ICS 214 Lab 6

Intro

Welcome to lab 6! This lab introduces classes and object-oriented programming along with a few other things we may not have encountered to date. This is the first of a series of 3 labs in which we are putting together a Tetris game. The next labs will build on top of this material.

Before starting, you should be familiar with the material in Module 11 which introduces OOP in C++.

Objectives:

1. Breaking classes into .h & .cpp files
2. Intro to building classes from UML diagrams
3. Accessibility specifiers (public, private, protected)
4. Vectors – creating/using.
5. Getters & setters
6. Using const with methods

In this lab we are using/creating two classes (Point and Tetromino) for use in Tetris. The classes built in this lab should account for roughly 25% of the code (line count) in the project. That being said – they are also the simplest classes in the project to build. Before we get started, there are a few things to mention: const, header/cpp files, vectors, and uml diagrams.

With classes, the terminology changes slightly. Previously we were writing functions. When functions belong to classes, they are often referred to as methods (or member functions).

Using const with methods

Something new that we are adding in this lab is the use of **const** after a **method** name. It isn't used for all methods – only those that don't modify the state (any variables) of the class.

```
Eg:  int getX() const
```

Labelling a method as const does a couple of things:

1. It tells anyone using your code that the method doesn't change the state of any member variables of the class.
2. It causes the compiler to complain if any changes are attempted within the method. This also prevents a method from calling any non-const methods.

Using const is not technically a requirement (by the compiler), but it is good "defensive programming", so we will incorporate it to use with the classes that we build in this project. To specify a method as const, we need to do so in both the header file (declarations) and in the cpp file (definitions) for any given class, otherwise the compiler will complain.

Header files and cpp files

In this lab we will be creating a header file and a cpp file for each class we define. Class declarations will be placed in the header file, and the class definitions in the cpp file.

When we write a class, we could theoretically write the entire class inside the class declaration in the .h file. However, doing this would result in all methods being. An inline function is an optimization that the compiler makes. When a function/method is inline, the compiler will copy and paste the contents of the function definition into every spot in code that it is called (rather than making the function call). This avoids the overhead of making a function call, but it results in object files that are much larger than they need to be. The compiler determines to make some functions inline based on a cost-benefit analysis it develops as it builds the project. The shorter the function, the more likely it is that the code will benefit from it being made inline.

Typically, the .h file should only contain class/function declarations (unless the methods are short enough to warrant being made inline). Many of the function definitions in Point.h are likely short enough to write inline. Let's use a "rule of thumb" and say that if the method can be written in a single line, we can inline it. Otherwise, put it in a .cpp file instead.

The.cpp files we create will need to have access to class declarations in the associated .h file. Note below how Point.cpp #includes Point.h.

Point.h

```
class Point {

public:
	int getX() const;
private:
	int x;
};
```

Point.cpp

```
#include "Point.h"

int Point::getX() const
{
	return x;
}
```

Notice how the class definition is placed in the .h file. The class's methods are declared inside the class (in the .h file), but the method definitions are placed inside the .cpp file (this should seem somewhat familiar). Next, we fill out our method definitions in the .cpp file. Prior to each method name (in the .cpp file), we specify the name of the class that owns the method so the linker knows which class the method definition belongs to.

We want to avoid using "using" statements in general, but especially within a header file. Why? If an unsuspecting programmer includes a header file you have written – they're probably not intending that an entire namespace be included & used along with it. This could potentially result in naming conflicts that are hard to track down.

Vectors:

The Tetromino class uses a std::vector of points instead of using an array. A vector is a data type similar to a fixed array – except that it is an object (it comes with a set of functions), and it is dynamically sized. Here are the basics that you will need for using a vector in this lab:

Creation/Initialization:

```
std::vector<Point> v1;
std::vector<Point> v2 { Point(0, 0), Point(1, 1)};
```

Assignment:

```
v1 = { Point(0, 0), Point(1, 1) };
v2 = { {0, 0}, {1, 1} };
```

Element Access (set/get):

```
v1[2] = Point(0, 0);
```

Adding a single element:

```
v1.push_back(Point(3, 3));
v2.push_back( {3, 3} );
```

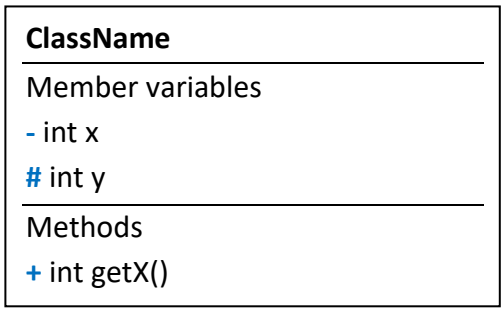
Emptying a vector:

```
v1.empty();
```

UML Diagrams

The UML (Unified Modelling Language) diagrams listed under the individual files below should be enough to get you started. These diagrams use a single character to specify the access for each item prior to the member variable/method:

- **public: +**
- **private -**
- **protected #**



Header Guards

Classes allows us to create custom **types**. Part 3 of the One Definition Rule (ODR) states that **types** are allowed to have identical definitions in different files. This is why we can place enumerations and class definitions (both are used to define types) in header files (since .h files are likely to be included in multiple files within the project).

It is possible to #include a header file multiple times in the same file which would break part 1 of the ODR that states "withing a given file, a **type** can only have one definition".
To better understand, imagine the following scenario:

- Two classes: B and C, use class A. Both B.h and C.h contain an #include "A.h" statement.
- Main.cpp uses both B and C, so it #includes both B.h and C.h.
- As a result, the preprocessor tries to #include "A.h" in main.cpp twice (since both B.h and C.h #include A.h, and the preprocessor deals with #include statements by copying and pasting the included statements directly into the source file.

The use of header guards (or #pragma once) in our .h files prevents this problem. Since classes and enums both add **type** definitions in our .h files, using header guards becomes even more important.

Assignment using "this":

```
void setX(int x) {
  x = x;   // both x's refer to the param
}
```

```
void setX(int x) {
  this->x = x;  // class x = param x
}
```

When we define a "setter" method for a class, often we want to assign a member variable a value passed into the function. If we named the formal parameter the same name as the member variable (with no other changes), we would run into trouble because when we specify x in the function – it always refers to the parameter.

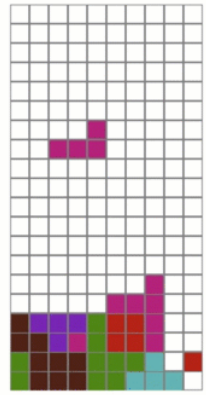
Can we somehow differentiate between the parameter and the class member variable? Yes! In C++, an instance of a class (an object) has a default member variable we can use called "this". This is a pointer that contains the memory address that the instance is stored at in memory. If our class is called "Employee", then the type of "this" will be Employee* (a pointer to an Employee). It follows that if we dereference this pointer, we can get the instance of the object itself.

We can use (*this).x , or this->x to get the x member variable of the class. Recall: both options will dereference the pointer to give us the object instance. The second method is just a bit simpler syntactically. More will be explained in future chapters, but for now it provides a simple way to differentiate between a member variable that has the same name as a parameter.

Only use the "this" pointer when it is required to distinguish between a parameter and a member variable. In the past I have noticed that sometimes the "this" keyword creeps into code in front of every single member variable. That is not the intent of "this" and it just clutters up the code.

Getting Started:

How do we get started writing Tetris? In this lab we'll take a "bottom up" approach and start by examining the smaller components that make up gameplay, and then trying to figure out a way to break those components down into bite sized chunks that can be represented by objects (classes). We can then try to figure out ways to represent the data within the classes using some of the different data types that we have at our disposal: arrays, vectors, enums, ints, etc…



So, what are the components that make up Tetris? The game is concerned with placing different "tetrominos" (each one containing 4 blocks) on a gameboard. A tetromino can be moved around, but when it collides with the bottom (or a block that has previously been placed on the board) its blocks become part of the gameboard.

A tetromino can have a variety of different shapes and colors. Th tetromino seems like a good candidate for a class because it represents a self contained, reusable object. How could we represent a tetromino? Each tetromino has a specific shape, and a specific color. So, to represent a tetromino we want to create a class with a shape, a color, and a way to represent the layout of the blocks (a different layout for each shape).

That leads to the question –"what is a block?" A block can be part of a tetromino, or it can be placed on the gameboard. How do we differentiate between one block and another? A block differs from another block by its color. We can use a scoped enumeration (enum class) for the different available colors.

What types of objects would "own" blocks? A tetromino would… so would a "gameboard". Each tetromino would be responsible for 4 blocks. A gameboard could be responsible for all the blocks that have been placed on the board. We'll look at the concept of a gameboard more in upcoming labs. To represent the different shapes of a tetromino, we would need a way to store the placement of blocks in relation to each other. Could we use a list of 2D points (on an XY co-ordinate system) for this? You bet!

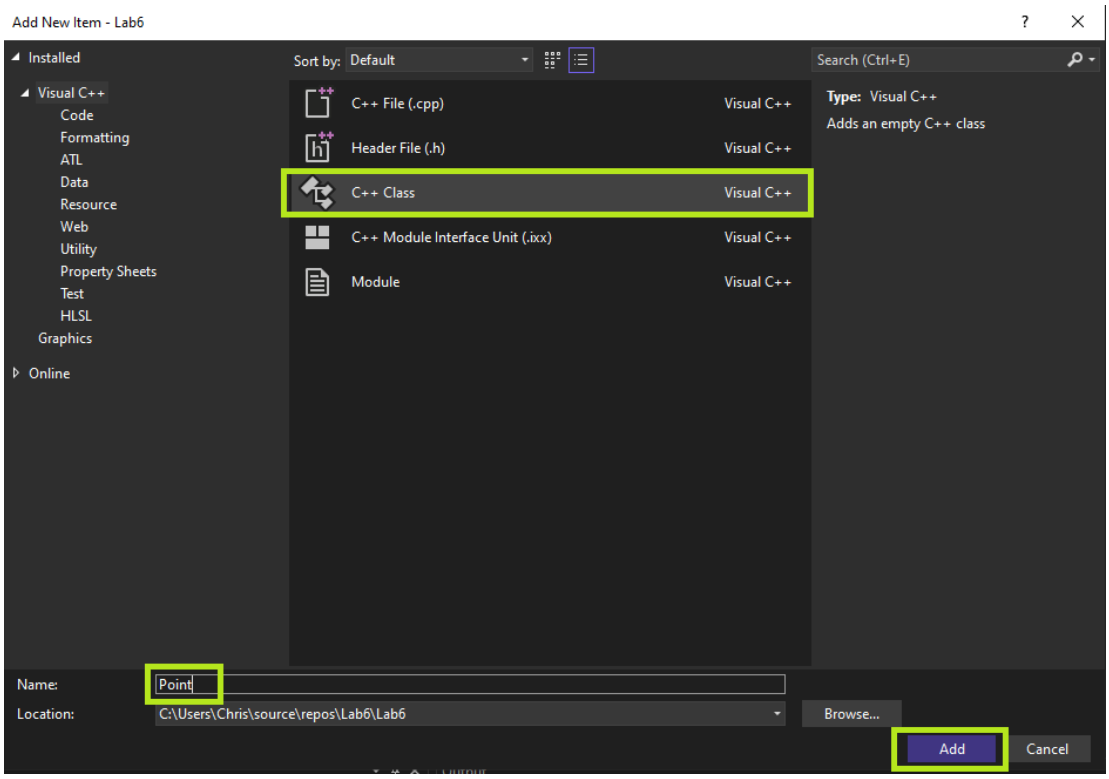
A 2D point is an XY co-ordinate. It is basic, self-contained, and could potentially be reused in a lot of projects, so it is a perfect candidate for encapsulating in a separate class. We need variables for storing x and y points, and we can add some methods for manipulating these points. That's all we need it to be. Nice and simple!

Since a tetromino can represent a number of different "shapes" (often named after the letters they look like), we can create a scoped enumeration to represent these. To fully define a tetromino, we need the concept of shape, color, and a list of points representing placement on a co-ordinate system. These three properties seem perfect for encapsulating in a tetromino class. We can anticipate later adding methods for rotating and displaying a tetromino.

Our focus for this lab will be on building a Point class and a Tetromino class. In the next lab we will work through building a Gameboard class.

First Steps

- Create a new empty project called Lab6.
- Create a Source.cpp file and write a main function as you would normally. We will use this for testing later.
- Download TestSuite.h and TestSuite.cpp from D2L (in the same folder as this project) and copy it into your project. Place it in your header files folder (View | Solution Explorer). This file will help you to test your classes prior to completing a demo. We'll step through this later in the lab.
- Create source files for Point.h & Point.cpp. Create source files for Tetromino.h & Tetromino.cpp.
    - Tip: if you're using VS2022, you can create both .h and cpp files together. Right click in the Solution Explorer on your source files directory, select "Add", select "New Item". On the dialog that appears (see below), select "C++ Class" and name it "Point". Click "Add". Click "OK" on the Add Class dialog that appears next. This will automatically create matching .h and .cpp files for a new Point class.



-
- Complete these new source files in with the info contained below.

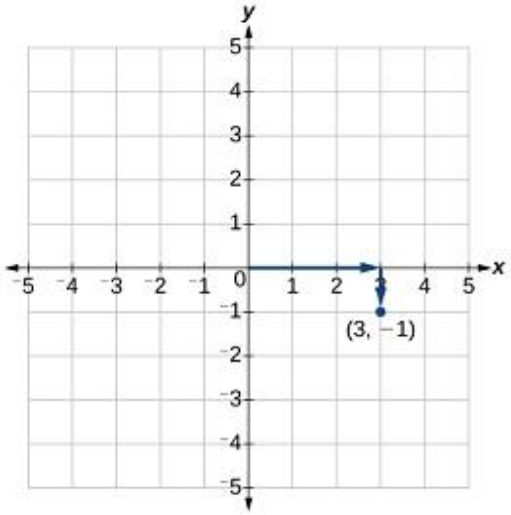
Classes for Point and Tetromino

Point Class

The point class encapsulates an x, y location (on a co-ordinate system).

It provides:

- setters and getters for accessing x, y.
- methods for any math operations that might be required in the game.



Point
- int x - int y
+ Point() + Point(int x, int y) + int getX() + int getY() + void setX(int x) + void setY(int y) + void setXY(int x, int y) + void swapXY() + void multiplyX(int factor) + void multiplyY(int factor) + std::string toString()

Create a Point class declaration inside Point.h. Here are some method declarations to get you started. Copy and past this directly into your .h file.

```
// swap x and y
void swapXY();

// multiply x by some factor
void multiplyX(int factor);

// multiply y by some factor
void multiplyY(int factor);

// return a string in the form "[x,y]" to represent the state of the Point instance
// (for debugging) You could use stringstream if you want to try a forming a string in
// the same way you use cout https://www.learncpp.com/cpp-tutorial/stream-classes-for-strings/
std::string toString() const;
```

[expected .cpp size is ~65 lines]

*See note on last page of doc if the stringstream link is not working.

Tetromino Class



Tetrominos are the block groupings used in every known Tetris game. They come in seven shapes and each (typically) has a unique color. Tetrominos all have four square blocks. The seven tetrominoes are named after the letters that they resemble.

The Tetromino class represents one of the shapes we see in Tetris

- 7 shapes are represented (defined in the enum class TetShapes).
- Each shape is comprised of 4 block locations (represented by a vector of Points)
- Each shape has a unique color associated with it (defined in enum class TetColor)

Tetromino
- TetColor color
- TetShape shape
- std::vector<Point> blockLocs
+ Tetromino()
+ TetColor getColor()
+ TetShape getShape()
+ void setShape(TetShape shape)
+ void rotateClockwise()
+ void printToConsole()

TetColor
RED
ORANGE
YELLOW
GREEN
BLUE_LIGHT
BLUE_DARK
PURPLE

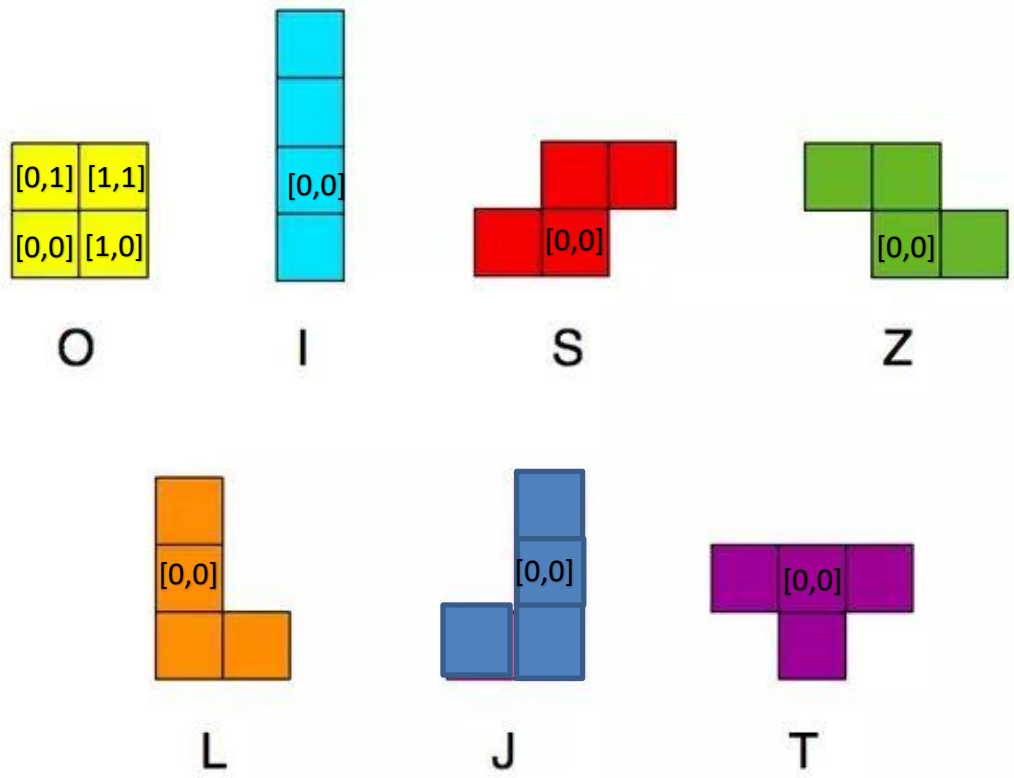
TetShape
S
Z
L
J
O
I
T

Enter the scoped enumeration for TetColor and TetShape in the Tetromino.h file, and add the Tetromino class declaration below them. You will need to use ALL CAPS (as shown above) for the entries in order to allow for automated testing to work later on.

A sizeable part of completing the Tetromino.cpp class is filling in the setShape() method to set the blockLocs member variable (a vector) to contain a different set of points when a given TetShape is passed in. The points that you will need to set it to can be determined from the figures below. Ensure you use the point(0,0) specified on each shape and determine the other points relative to it (otherwise your shape will not rotate around 0,0).

The rotateClockwise() method needs to transform the points by rotating them around the point [0,0]. To complete this function, you will need to figure out what sort of math operation(s) it would take to transform a point 90 degrees clockwise on a co-ordinate system (draw it out on graph paper to get a visual on this). Eg: Point(1,2) rotates clockwise to Point(2, -1) etc…

Note that the tetromino block locations are based on a standard co-ordinate system that is the same as the one shown in the Point class above.

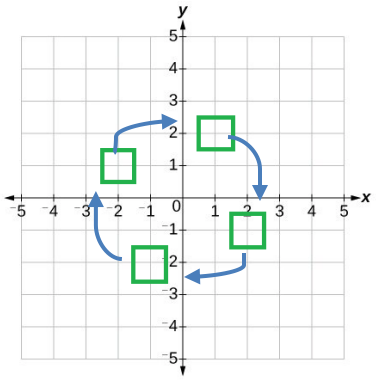


Here are some details that might not be obvious. Copy and past the following code into your .h & .cpp files to get you started.

```
// - set the shape
// - set the blockLocs for the shape
// - set the color for the shape
void setShape(TetShape shape);//

// print a grid to display the current shape
// to do this: print out a "grid" of text to represent a co-ordinate
// system.  Start at top left [-3,3] go to bottom right [3,-3]
// (use nested for loops)
//  for each [x,y] point, loop through the blockLocs and if the point exists
// in the list, print an 'x' instead of a '.'. You should end up with something
// like this: (results will vary depending on shape and rotation, eg: this
//  one shows a T shape rotated clockwise once)
//  .......
//  .......
//  ...x...
//  ..xx...
//  ...x...
//  .......
void printToConsole();

// rotate the shape 90 degrees around [0,0] (clockwise)
// to do this:
//  - iterate through blockLocs
//  - rotate each Point 90 degrees clockwise around [0,0]
// hint: as you rotate the point [1,2] clockwise around [0,0], note
// how the x,y values change. Once you find the pattern you can use
// a couple existing methods in the Point class to accomplish this.
// make it so that the TetShape::O doesn't rotate
void rotateClockwise();
```



[expected .cpp size is ~100 lines]

Add const to methods that don't modify internal state

When a const object is passed to a function or method by const reference, it is only able to use its methods that have been labelled as const. Rightly so… this ensures that the internal state of the const object wont' change!

Go through all the methods you have just written for Point and Tetromino, identify the ones that don't change the internal state (the member variables) of the object instance, and label them as const. This will allow anything being passed by const reference to have access to as much functionality as possible. This concept is covered in LearnCpp section 13.2 (const class objects and member functions).

Automated testing with TestSuite.h

This supplied file allows you to self-test the development of some of the classes that you will write. Once you have your classes built and are ready to test them, add the following at the top of Source.cpp (or whatever cpp file your main() is in):

```
#include "TestSuite.h"
```

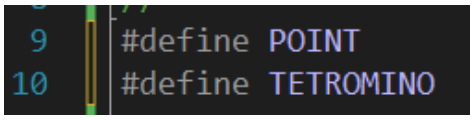
Add this as the first line in your main function():

```
// run some sanity tests on our classes to ensure they're working as expected.
TestSuite::runTestSuite();
```

Open the header file of the class you are testing and add this line inside the class definition to allow TestSuite access to private variables & functions of the class:

```
// FRIENDS
friend class TestSuite;// (allows TestSuite access to private members for testing)
```

You will also need to uncomment the #define statements in TestSuite.h



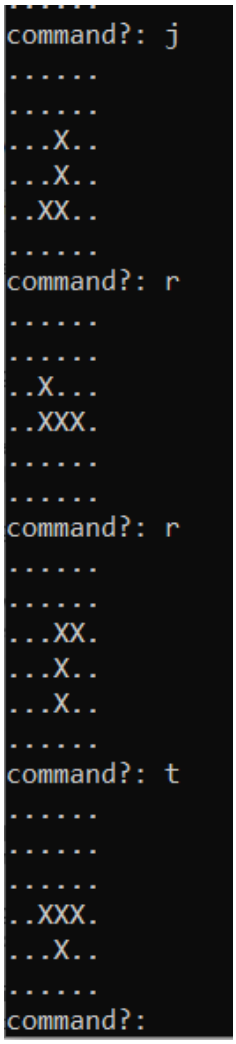
This will run some sanity checks on the basic functionality class you have written when you try to compile and run. The tests are in no way comprehensive, but the hope is that they will point to some errors that might have otherwise been less apparent or harder to produce.

The test suite makes use of a lot of assert statements. If a boolean assert statement equates to false, the program will cease execution immediately and leave a message pointing you to the line in the TestSuite that failed. This should allow you to examine the line causing the issue and give you some insight into the problem.

Write a Testbed for the Demo

You're almost done. Once you have completed the Point and Tetromino classes, write a little testbed in main() to demonstrate things work as you expect.

- Instantiate a Tetromino in your main function.
- Write a mechanism (probably a while loop which uses cin to accept a char of user input) to allow you to set and rotate the tetromino. Use 'r' to rotate, and the associated shape character to change the shape (ie: entering 's' would change to the s shape). After each change, use the printToConsole() method to verify the correct change has taken place on the console. This should take about 5-10 mins.
- Use this mechanism to prove to yourself that ALL your shapes and rotations work. It will be used to demo your code as well
- Output should look similar to the following:



Marking: [out of 10]

Marking will be done via a demo (play-testing) and a code walkthrough.

- Does it work as expected?
- Variables: Have good variable & const names been used?
- Are variables in local scope instead of global?
- Have constructors used initialization lists when appropriate (instead of assignment)?
- Are there comments to explain anything non-trivial or not obvious? Is there at least 1 comment at the top of every function (in the header file)?
- Have variables been initialized with best practices (uniform initialization).
- Have header guards (or #pragma once been added to .h files?

Change Log:

Oct 18, 2023	Updated the outdated web link on how to use stringstream.
	Allowed students to use inline methods for methods of one line.

Oct 12, 2024	I noticed the stringstream web link (learncpp section 28.4) is showing an error (establishing database connection). Hopefully this is temporary and is fixed by the time you see it. If not – here's a short example of how you can use it to format and output strings.
	
	#include <sstream>
	#include <iostream>
	void formatMessage(const std::string& name, int age) {
	std::stringstream ss;
	ss << "Hello, " << name << ". You are " << age << " years old.";
	std::cout << ss.str() << std::endl;
	}
	
	[source: https://medium.com/@burakroccia/mastering-stringstream-in-c-a-comprehensive-guide-7eda8a47679a]